Query Processing
Evaluation of Expressions

---

1. Introduction

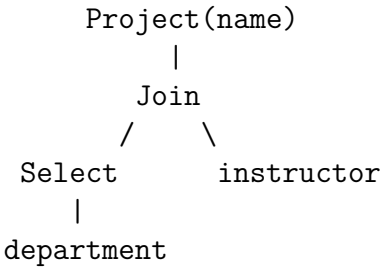
A query typically contains **multiple operations** (selection, join, projection, etc.). Two main strategies exist for evaluating such expressions:

Approach	Description
Materialization	Execute one operation at a time; write results to temporary relations
Pipelining	Pass results directly between operations without storing intermediates

Example Expression

$$\Pi_{name}\left(\sigma_{building=\text{"Watson"}}(department) \bowtie instructor\right)$$

Operator Tree:



2. Materialization

Strategy: Evaluate operations bottom-up in the operator tree, storing each intermediate result as a **temporary relation** on disk.

Algorithm: Materialized Evaluation

1. Start at the lowest-level operations (leaves of tree)
2. Execute each operation using appropriate algorithm
3. Store result in temporary relation on disk
4. Use temporary relations as input for next-level operations
5. Repeat until root operation produces final result

Materialization Example

For: $\Pi_{name}(\sigma_{building=\text{"Watson"}}(department) \bowtie instructor)$

1. Execute $\sigma_{building=\text{"Watson"}}(department) \rightarrow$ Write to temp1
2. Execute $temp1 \bowtie instructor \rightarrow$ Write to temp2
3. Execute $\Pi_{name}(temp2) \rightarrow$ Final result

Cost of Materialization

Total cost = Sum of all operation costs + **Cost of writing intermediate results**
Writing cost per intermediate result:

$$\text{Block Transfers} = \left\lceil \frac{n_r}{f_r} \right\rceil = b_r$$

$$\text{Seeks} = \left\lceil \frac{b_r}{b_b} \right\rceil$$

Where:

- n_r = number of tuples in result
- f_r = blocking factor (tuples per block)
- b_b = output buffer size in blocks

Double Buffering: Use two output buffers — one writes to disk while the other accumulates tuples. This allows CPU and I/O to work in parallel, improving performance.

3. Pipelining

Strategy: Pass tuples directly from one operation to the next **without writing to disk**.

Benefits:

1. **Eliminates I/O cost** of intermediate results
2. **Faster response time** — results start appearing immediately
3. **Lower memory usage** — no need to store full intermediate relations

Pipelining Example

For: $\Pi_{a1,a2}(r \bowtie s)$
Without Pipelining:

1. Compute $r \bowtie s$ completely $\rightarrow$ Write to disk
2. Read result from disk $\rightarrow$ Apply projection

With Pipelining:

1. As join produces each tuple, immediately pass to projection
2. Projection outputs final result directly
3. No intermediate storage!

3.1 Demand-Driven Pipeline (Pull Model)

The **consumer** requests tuples from the **producer**.

Iterator Interface

```
Each operator implements three functions:

open()  - Initialize the operation; open inputs
next()  - Return the next output tuple
close() - Clean up; release resources

State is maintained between next() calls.
```

Iterator Example: Selection with Linear Scan

```
class SelectIterator:
    state: current_position_in_file

    open():
        start file scan at beginning
        state.position = 0

    next():
        while not end_of_file:
            tuple = read_next_tuple()
            state.position++
            if tuple satisfies predicate:
                return tuple
        return NULL  // no more matches

    close():
        close file scan
```

Iterator Example: Join Calling Selection

```
class JoinIterator:
    open():
        left_input.open()
        right_input.open()

    next():
        // Get tuple from left (which may be a SelectIterator)
        left_tuple = left_input.next()
        // Match with right input...
        return matched_tuple

    close():
        left_input.close()
        right_input.close()
```

Key Point: Higher-level operators call `next()` on lower-level operators, which in turn call `next()` on their inputs — a chain of pulls from root to leaves.

3.2 Producer-Driven Pipeline (Push Model)

The **producer** generates tuples eagerly and pushes them to the **consumer**.

Producer-Driven Architecture

```
Each operator runs as a separate process/thread:

1. Producer generates tuples continuously
2. Puts tuples in output buffer
3. When buffer is full, producer WAITS
4. Consumer takes tuples from buffer
5. When buffer has space, producer resumes

Buffers connect adjacent operators in the pipeline.
```

Aspect	Demand-Driven (Pull)	Producer-Driven (Push)
Control	Consumer requests	Producer generates eagerly
Implementation	Iterator functions	Separate threads + buffers
Common Use	Most query engines	Parallel systems, streaming
Efficiency	More function calls	Fewer function calls

4. Blocking vs. Pipelined Operations

Not all operations can be fully pipelined.

Edge Type	Meaning
Pipelined Edge	Tuples flow immediately between operators
Blocking Edge	Output cannot start until all input is consumed

4.1 Examples of Blocking Behavior

Operation	Blocking Behavior
Sorting	Blocks between run-generation and merge phases
Hash Join	Blocks until both inputs are partitioned; build-probe can then pipeline output
Aggregation	May block until all groups are formed (unless on-the-fly)
Indexed Nested-Loop	Pipelined on outer; blocking on inner (index must exist)
Merge Join	Fully pipelined if both inputs are already sorted

Hash Join: 3-Phase Breakdown

Hash join can be modeled as three sub-operators:

1. **Partition** r: Pipelined input from r; blocking output
2. **Partition** s: Pipelined input from s; blocking output
3. **Build-Probe**: Blocking input (waits for partitions); pipelined output

```
[Scan r] --pipelined--> [Partition r] --blocking-->
                                                    [Build-Probe] --pipelined-->
[Scan s] --pipelined--> [Partition s] --blocking-->
```

4.2 Pipeline Stages

A **pipeline stage** is a maximal set of operators connected only by pipelined edges.

1. All operators in a stage execute **concurrently**
2. Stages are separated by **blocking edges**
3. Query executes one stage at a time

5. Double-Pipelined Join

A special algorithm that pipelines **both inputs** simultaneously.

Use Case: When both input streams arrive concurrently (e.g., from sub-queries).

Algorithm: Double-Pipelined Hash Join

```
done_r = false; done_s = false
r = {}; s = {}  // In-memory with hash indices
result = {}
queue = shared input queue for tuples from r and s

while not (done_r AND done_s):
    wait until queue is not empty
    t = dequeue()

    if t == End_r:
        done_r = true
    else if t == End_s:
        done_s = true
    else if t is from r:
        r = r union {t}
        update hash index on r
        result = result union ({t} join s)  // Probe s with new r tuple
    else:  // t is from s
        s = s union {t}
        update hash index on s
        result = result union (r join {t})  // Probe r with new s tuple
```

Key Features:

- Both inputs processed as they arrive
- Hash indices on both r and s for fast probing
- Output produced incrementally

Memory Overflow Handling:

1. Process tuples in-memory until memory is full
2. Treat in-memory tuples as partition 0 (r_0, s_0)
3. New tuples go to partition 1 (r_1, s_1) on disk
4. Before writing to disk, probe against in-memory partition
5. After all input, join $r_1 \bowtie s_1$ using standard hash-join

6. Pipelines for Continuous Streams

For **data streams** (e.g., sensor data, log events), queries run continuously.

Characteristics:

- Data arrives continuously, not stored in traditional tables

- Queries are **continuous queries** — always running
- Results output as data arrives
- Producer-driven pipelines are ideal

6.1 Windowed Aggregation

Most stream queries use **tumbling windows** — fixed time intervals.

Stream Aggregation Example

Query: Average temperature per minute from sensor stream

```
SELECT window_start, AVG(temperature)
FROM sensor_stream
GROUP BY TUMBLING_WINDOW(1 minute)
```

Process:

1. As tuples arrive, add to current window's hash table
2. Compute running sum and count
3. When window closes (1 minute passes):
   - Output (window_start, sum/count)
   - Start new window

6.2 Window Completion Detection

How do we know when a window is complete?

Method	Description
Ordered Tuples	Tuple from next window indicates previous is complete
Punctuations	Special markers: "No more tuples with timestamp $< X$"

7. Summary

Approach	Pros	Cons
Materialization	Simple; any algorithm works	High I/O for intermediates
Pipelining	No intermediate I/O; fast response	Not all ops can pipeline
Demand-Driven	Easy to implement	More function call overhead
Producer-Driven	Efficient; good for parallelism	Complex; needs thread management

Key Takeaways

- **Materialization:** Execute operators bottom-up; write intermediates to disk
- **Pipelining:** Pass tuples directly; eliminates intermediate I/O
- **Iterator Model:** `open()`, `next()`, `close()` — demand-driven pipelining
- **Blocking operations** (sort, hash-join partitioning) break pipelines into stages
- **Double-pipelined join:** Both inputs streamed; output produced incrementally
- **Continuous queries:** Producer-driven pipelines + windowed aggregation
- **Pipeline stages:** Groups of operators separated by blocking edges
- **Hybrid hash-join:** Partially pipelined if first partition fits in memory